

LLM Mental Model & API Setup

Understand how Large Language Models work — and how to start building with them.

Module 1 — Foundations Track

Covers: Tokens · Context Windows · Sampling · Statelessness

OpenAI & Gemini Setup · Anatomy of an API Call · Prompt Reasoning

Contents

1. How LLMs Actually Work

2. Tokens & Tokenization

3. Context Windows

4. Statelessness

5. Sampling & Generation Parameters

6. Setting Up OpenAI & Google Gemini

7. Anatomy of an API Call

8. LLMs as Programmable Reasoning Engines

9. Glossary — Quick Reference

10. Final Exam-Style Checklist

01 How LLMs Actually Work

Think of an LLM as an extremely well-read autocomplete engine: given everything it has seen so far, it predicts the single most probable next token — over and over — until a full response emerges.

Core Mechanism

- **Next-token prediction** is the only thing the model fundamentally does. Every "capability" (reasoning, coding, chatting) is an emergent behavior of this one operation repeated thousands of times.
- The model outputs a **probability distribution** over its entire vocabulary for the next token, then a token is selected (see Sampling, Section 5).
- There is no "thinking" between tokens by default — each token is generated in a single forward pass through the network.

Training Lifecycle

Stage	What Happens	Goal
Pretraining	Predict next token on massive raw text corpora	Learn language, facts, patterns
Instruction Tuning (SFT)	Fine-tune on curated (prompt → ideal response) pairs	Learn to follow instructions
RLHF / Preference Tuning	Reward model ranks outputs; policy optimized against it	Align tone, safety, helpfulness

Why Models Hallucinate

- The model is optimizing for *plausible* continuations, not verified truth — it has no built-in fact-checker.
- Confidence in tone is unrelated to factual confidence — a hallucinated answer can sound exactly as fluent as a correct one.
- Hallucination risk increases with: obscure facts, long numerical outputs, and prompts that presuppose false premises.

PITFALL

Assuming a fluent, confident-sounding answer is a correct one. Always verify facts, citations, and numbers independently when accuracy matters.

EXAM ANGLE

"Explain why an LLM can produce a wrong answer with high fluency." → Because fluency comes from token-probability modeling, not from a ground-truth lookup process.

02 Tokens & Tokenization

What Is a Token?

A token is the atomic unit an LLM reads and writes — usually a subword chunk, not a full word.

"unbelievable" might split into ["un", "believ", "able"] .

KEY FACTS

- Rule of thumb: **~4 characters ≈ 1 token** in English; **~0.75 words ≈ 1 token**.
- Tokenizers are algorithmic (BPE — Byte Pair Encoding, or SentencePiece), trained once on a text corpus, and fixed per model family.
- Different providers use different tokenizers — the same string will not cost the same number of tokens on GPT vs. Gemini.
- Non-English languages and code often tokenize **less efficiently** (more tokens per character) than English prose.

WHY TOKENIZATION MATTERS

Impact Area	Explanation
Pricing	API cost is billed per input + output token, not per word or character
Context Limits	The context window is measured in tokens, not words
Weird Behavior	Models are sometimes bad at character-level tasks (e.g. counting letters) because they never "see" individual letters — only token chunks

TIP

Use a tokenizer visualizer (e.g. OpenAI's "Tokenizer" tool) early on — seeing how your actual prompts get split builds real intuition fast.

Tokens are like LEGO bricks of language: the model doesn't see whole sentences or words, only a stream of these standardized pieces, and it learns statistical patterns of which bricks tend to follow which.

03 Context Windows

Definition

The context window is the maximum number of tokens (input + output combined) the model can "see" and reason over in a single call.

PRACTICAL IMPLICATIONS

- Anything outside the window is **invisible** to the model — it's not "forgotten," it was simply never included.
- Larger windows cost more compute → generally **higher latency and price** per call.
- **"Lost in the middle"** effect: models tend to attend better to information at the start and end of a long context than content buried in the middle.
- System prompt + conversation history + retrieved documents + the model's own output **all share the same budget**.

Strategy	When to Use
Truncate oldest messages	Simple chat apps with linear history
Summarize older turns	Long-running assistants that need continuity
Retrieval (RAG)	Large knowledge bases too big to fit in-context
Sliding window	Real-time streams (chat, logs) needing only recent context

PITFALL

Silently exceeding the context window causes the oldest content to be dropped or the call to error out — always track token usage programmatically rather than assuming it'll "just fit."

04 Statelessness

The Core Idea

An LLM API call has **zero memory** of any previous call. Every request is evaluated fresh, with no hidden session state on the model's side.

Calling an LLM API is like talking to someone with total amnesia after every sentence — if you want them to "remember" the conversation, you must hand them a full written transcript every single time.

HOW CHAT APPS FAKE MEMORY

- The client (your app) stores the full message history locally or in a database.
- On every new turn, the **entire conversation** (system + user + assistant messages) is resent as the `messages` array.
- The model re-reads everything from scratch each time — this is why long conversations get slower and pricier per turn.

DESIGN IMPLICATIONS

- You are responsible for: storage, truncation, summarization, and retrieval of past context.
- "Memory" features in consumer products (e.g. persistent user facts) are engineered on top of the stateless API — not native to the model.
- This is also why prompt-injection or leaked history in one call can't "carry over" and corrupt a completely separate session — each call is isolated.

EXAM ANGLE

"How does ChatGPT remember earlier messages if the model is stateless?" → The app resends the full (or summarized) conversation history with every API call; the model itself retains nothing between calls.

05 Sampling & Generation Parameters

From Probabilities to Text

After the model computes a probability distribution over the next token, a **decoding strategy** decides which token actually gets picked.

KEY PARAMETERS

Parameter	What It Controls	Typical Range
<code>temperature</code>	Randomness of sampling. 0 = deterministic/greedy, higher = more diverse/risky	0.0 – 2.0
<code>top_p</code> (nucleus sampling)	Restricts sampling to the smallest set of tokens whose cumulative probability $\geq p$	0.1 – 1.0
<code>top_k</code>	Restricts sampling to the k most likely tokens	1 – 100
<code>max_tokens</code>	Hard cap on output length	model-dependent
<code>stop</code>	Sequence(s) that halt generation immediately when produced	custom strings
<code>seed</code>	Fixes the random number generator for more reproducible output	integer

DECODING STRATEGIES

- **Greedy decoding** — always pick the single highest-probability token. Deterministic but can be repetitive/dull.
- **Sampling** — pick probabilistically according to the (possibly temperature/top-p adjusted) distribution. More natural and varied, but non-deterministic.

TIP

Low temperature (0–0.3) for factual/deterministic tasks (data extraction, code); higher temperature (0.7–1.0) for creative brainstorming and varied writing.

PITFALL

Setting `temperature` and `top_p` both to extreme, conflicting values (e.g. `temperature=2`, `top_p=0.1`) — they interact, so tune one at a time.

PITFALL

Even with `temperature=0` and a fixed `seed`, outputs are not always perfectly reproducible across model versions or provider infrastructure — treat "deterministic" as "mostly deterministic."

06 Setting Up OpenAI & Google Gemini

OpenAI Setup

1. Create an account at platform.openai.com (separate from a ChatGPT Plus subscription — API billing is independent).
2. Navigate to **API Keys** and generate a new secret key — it is shown only once, so store it immediately.
3. Set up billing under **Settings** → **Billing**; add a payment method and, ideally, a usage budget/alert.
4. Note your **usage tier** — new accounts start on lower rate limits (requests/min, tokens/min) that increase automatically with spend history.
5. Explore the **Playground** to test prompts and parameters visually before writing code.

Google Gemini Setup

1. Two entry points exist — know the difference:
 - **Google AI Studio** (aistudio.google.com) — fastest way to get an API key, generous free tier, best for prototyping.
 - **Vertex AI** (Google Cloud) — enterprise-grade, tied to a GCP project, IAM permissions, better for production at scale.
2. In AI Studio: sign in with a Google account → **Get API Key** → create key in a new or existing project.
3. Free tier has strict rate limits (requests per minute/day) — check current limits before load testing.

TIP

Prototype in AI Studio for speed; migrate to Vertex AI only when you need enterprise auth, VPC controls, or SLA-backed production guarantees.

Key Security Practices

- ☐ Never hardcode API keys in source code or commit them to git
- ☐ Store keys in environment variables (`.env` files, excluded via `.gitignore`)
- ☐ Use separate keys per environment (dev / staging / prod)
- ☐ Set spend limits / budget alerts on both platforms
- ☐ Rotate keys immediately if one is ever exposed

07 Anatomy of an API Call

Request Structure

```
// Example: OpenAI-style chat completion request
POST https://api.openai.com/v1/chat/completions
Headers:
  Authorization: Bearer YOUR_API_KEY
  Content-Type: application/json

Body:
{
  "model": "gpt-4.1",
  "messages": [
    { "role": "system", "content": "You are a helpful assistant." },
    { "role": "user", "content": "Explain tokens in one sentence." }
  ],
  "temperature": 0.7,
  "max_tokens": 200
}
```

MESSAGE ROLES

Role	Purpose
system	Sets behavior, persona, and constraints for the whole conversation
user	The human's input / question
assistant	The model's prior replies — included when resending history

Response Structure

```
// Simplified response
{
  "id": "chatcmpl-abc123",
  "choices": [
    {
      "message": { "role": "assistant", "content": "..." },
      "finish_reason": "stop"
    }
  ],
  "usage": {
    "prompt_tokens": 24,
    "completion_tokens": 18,
    "total_tokens": 42
  }
}
```

FINISH_REASON VALUES

Value	Meaning
<code>stop</code>	Model completed naturally or hit a stop sequence
<code>length</code>	Output was cut off by <code>max_tokens</code> — response may be incomplete
<code>content_filter</code>	Output blocked by safety filtering

Streaming vs. Non-Streaming

- **Non-streaming:** wait for the full response, then receive it as one JSON object.
- **Streaming:** tokens are sent incrementally (Server-Sent Events) — critical for responsive chat UIs.

Error Handling

Status	Meaning	Fix
401	Invalid/missing API key	Check key and header format
429	Rate limit exceeded	Exponential backoff + retry
500 / 503	Provider-side error	Retry with backoff; check status page

TIP

Always implement exponential backoff with jitter for 429/5xx errors instead of naive immediate retries — this is standard production practice.

08 LLMs as Programmable Reasoning Engines

The Mental Model Shift

Stop thinking of an LLM as a chatbot. Start thinking of it as a **function you program in natural language**: inputs (prompt + parameters) → outputs (text, structured data, or a tool call).

A well-designed prompt is like a function signature: the system message defines the "contract," the user message supplies the "arguments," and the parameters (temperature, max_tokens) tune execution behavior.

PROMPTING TECHNIQUES

Technique	Description
Zero-shot	Ask directly with no examples — relies purely on pretrained knowledge
Few-shot	Provide 2–5 examples of input → output pairs to steer format/style
Chain-of-thought	Ask the model to reason step-by-step before the final answer, improving accuracy on multi-step problems

STRUCTURED OUTPUTS

- Request strict **JSON output** for programmatic parsing (many providers support a dedicated JSON/structured-output mode).
- **Function/tool calling**: the model can output a structured "call this function with these arguments" object instead of free text — the foundation of agentic workflows.

RELIABILITY PRINCIPLES

- ☐ Use low temperature for deterministic/parseable tasks
- ☐ Always validate and error-handle model output before using it downstream (never trust blindly)
- ☐ Prefer explicit output schemas/JSON mode over asking nicely in prose
- ☐ Design prompts to fail gracefully (e.g. "if unsure, output null" rather than guessing)

EXAM ANGLE

"Why treat an LLM as a reasoning engine rather than a search engine?" → It doesn't retrieve stored facts; it computes a plausible answer via learned patterns — reliability comes from how you constrain the "function," not from an internal database.

09 Glossary — Quick Reference

Token

The atomic text unit (subword) an LLM processes.

Context Window

Max tokens (input + output) visible to the model in one call.

Temperature

Sampling randomness control; 0 = deterministic, higher = more varied.

Top-p (Nucleus Sampling)

Samples only from the smallest token set covering cumulative probability p.

Statelessness

The model retains no memory between separate API calls.

System Prompt

Instruction defining the model's persona, rules, and behavior for a session.

Hallucination

Fluent but factually incorrect or fabricated output.

RLHF

Reinforcement Learning from Human Feedback — aligns model outputs to human preferences.

finish_reason

Field indicating why generation stopped (stop / length / content_filter).

Function/Tool Calling

Model outputs a structured request to invoke an external function instead of plain text.

RAG (Retrieval-Augmented Generation)

Injecting retrieved external documents into the prompt to extend the model's effective knowledge.

10 Final Exam-Style Checklist

Before moving to Module 2, you should be able to answer all of these without notes:

- ☐ Explain, in one sentence, what an LLM computes at each generation step.
- ☐ Describe why token count differs from word count, and why it matters for cost.
- ☐ Explain what happens when a conversation exceeds the context window.
- ☐ Explain how a stateless model can power a multi-turn chat app.
- ☐ Compare temperature vs. top_p and when you'd tune each.
- ☐ List the three message roles and what each is for.
- ☐ Name three `finish_reason` values and what each implies.
- ☐ Explain the difference between Google AI Studio and Vertex AI.
- ☐ Describe zero-shot vs. few-shot vs. chain-of-thought prompting.
- ☐ Explain why an LLM should be thought of as a reasoning engine, not a search engine.

"Master the fundamentals — everything you build later stands on this module."